Architecture des microprocesseur

Module SLEEL602

Cours de deuxième semestre

L3 Electronique

Pré-requis : module SPUE42

François VERDIER

Laboratoire LEAT

Université Côte d'Azur

- Ces enseignements ont pour objectif de vous apprendre à programmer correctement les microprocesseurs 8 bits actuels en assembleur
- Ces microprocesseurs sont aujourd'hui quasiment tous de type RISC (à jeu d'instruction réduit), pipeline (ils enchaînent plusieurs instructions à la fois) et extrêmement rapides
- L'objectif de ce cours est de vous enseigner ce que sont exactement les microprocesseurs et leur programmation

- Les cours de ce module feront 8 heures, donc seront répartis sur 6 séances de 1,5 heure
- Les TD de ce module seront répartis en 6 séances de 1,5 heure (8 heures)
- Les travaux pratiques de ce module seront de 3 séances de 3 heures (9 heures)

Bibliographie

V.C. Hamacher, Z.G. Vranesic, S.G. Zaky. «Computer Organization». Mac Graw-Hill, 2002.

D.A. Patterson, J.L. Hennessy. «Organisation et conception des ordinateurs : l'interface matérielle/logicielle». Dunod, 1994.

D.A. Patterson, J.L. Hennessy. «Architecture des Ordinateurs : une approche quantitative». Vuibert, 2003.

W. Stallings. «Computer organization and architecture: designing for performance». Prentice Hall, 2009.

Plan du cours

1. La programmation en assembleur
2. Les machines de calcul
3. Les branchements conditionnels
4. Le micro-contrôleur ATmega2560
5. Les interruptions
6. Les périphériques

Plan du cours

1. La programmation en assembleur
2. Les machines de calcul
    1. Les machines à pile
    2. Les machines à accumulateur
    3. L'algorithme de rétro-calcul
3. Les branchements conditionnels
    1. Le registre CCR
    2. Les instructions de branchement
    3. Quelques exemples de programmes
4. Le micro-contrôleur Atmega2560
    1. Les micro-contrôleurs AVR Atmega2560
    2. Les ressources internes du CPU AVR
    3. Le jeu d'instruction de l'AVR Atmega2560
    4. Le brochage de la carte Arduino ATmega2560
5. Les interruptions
    1. Les branchements aux sous-programmes
    2. Les interruptions
6. Les périphériques
    1. Les ports d'entrée/sortie parallèles
    2. Le convertisseur analogique/numérique

Chapitre 1 :
La programmation en assembleur

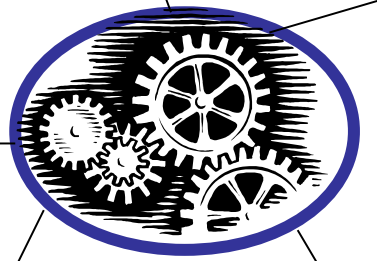
Ordinateur

écran



périphériques





Ordinateur



clavier

Lignes de
communication



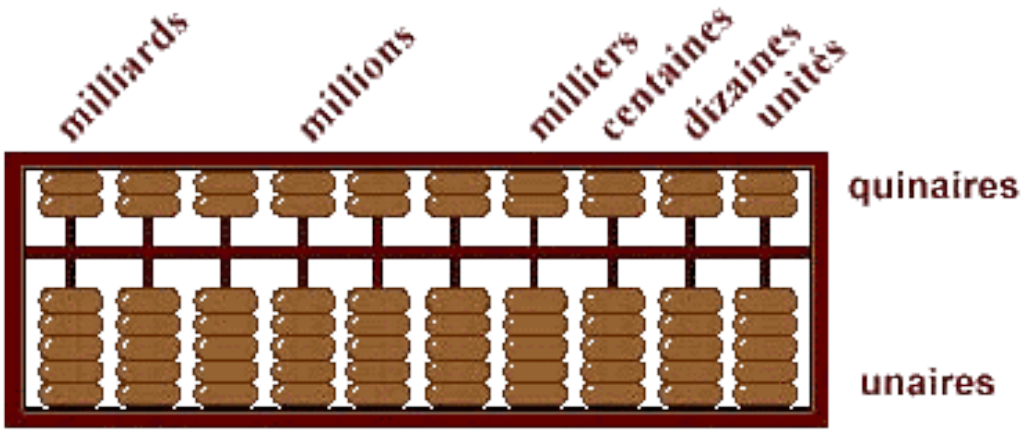


Introduction (1)

- A et B étant des entiers, on veut mettre dans C la différence entre A et B.
- Algorithme:
  - Lire A, lire B
  - Calculer A-B
  - Ecrire le résultat dans C

Représenter/Mémoriser

Vous avez déjà étudié les représentations binaires ...



Calculer

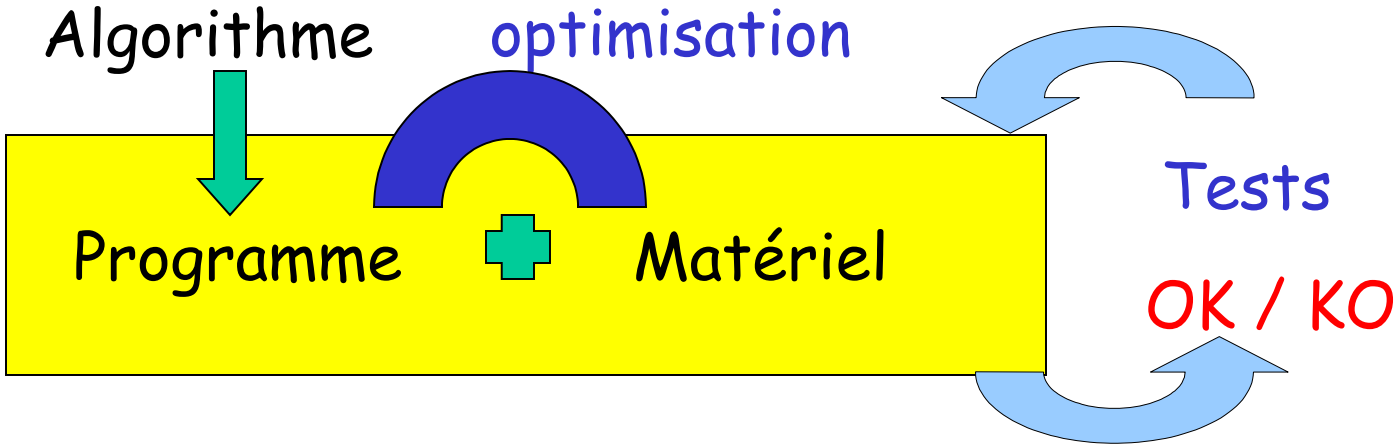
- Doigts
- Boulier
- Calculette
- Unité Arithmétique

il manque la notion de

programme enregistré

Introduction (2)

- La machine vue par l'utilisateur (programmeur)



- Langage de haut-niveau + compilateur
- Langage assembleur

Instruction

Informations associées à une instruction :

1. Opération à effectuer
2. Opérandes données (entrée)
3. Opérandes résultats (sortie)
4. Continuation (instruction suivante)

Problème :

Certaines info sont implicites ou inexistantes

Classification des Instructions

Où sont les opérandes ?

1. Mémoire
   - Principale
   - *Virtuelle*
   - Pile
2. UC
   - Registres généraux
   - Accumulateur
3. *Dispositif d'E/S*

Chapitre 2 : les machines de calcul

1. Les machines à piles
2. Les machines à accumulateur
3. L'algorithme de rétro-calcul

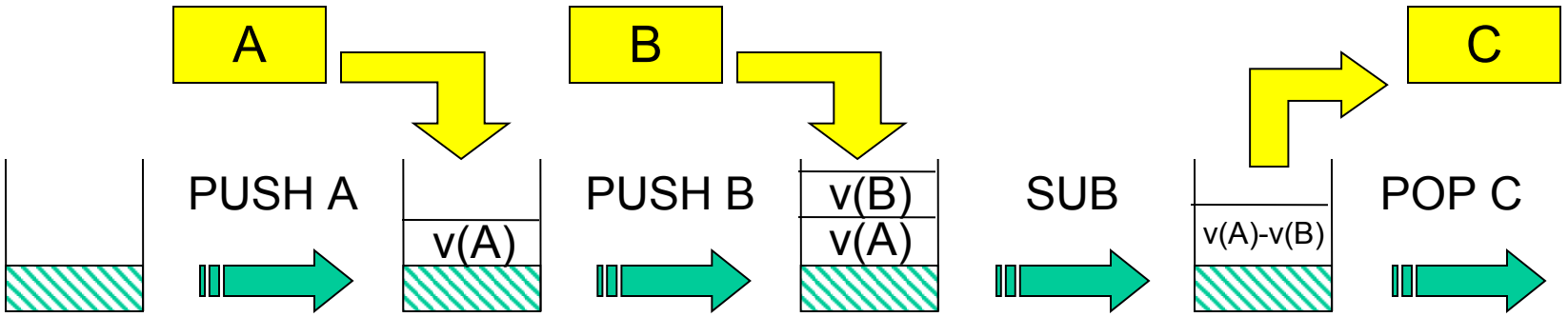
Chapitre 2 : les machines de calcul

1. Les machines à piles
2. Les machines à accumulateur
3. L'algorithme de rétro-calcul

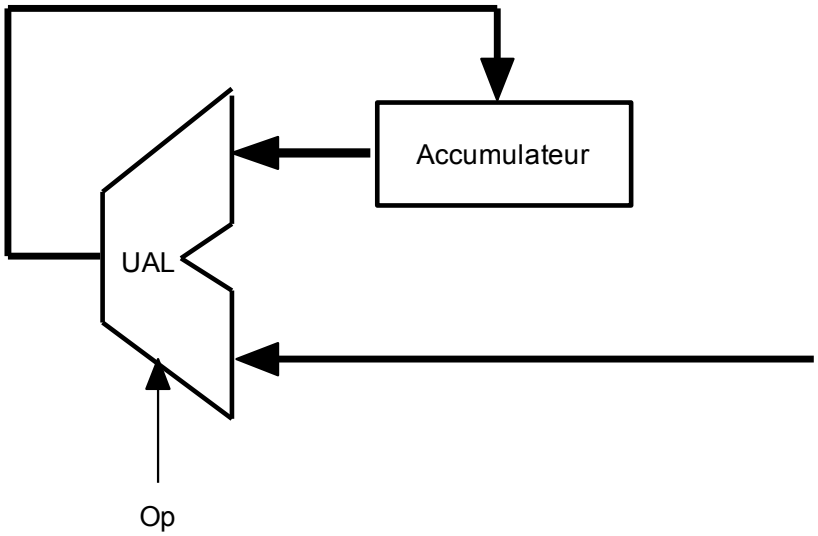
Les Machines à Pile

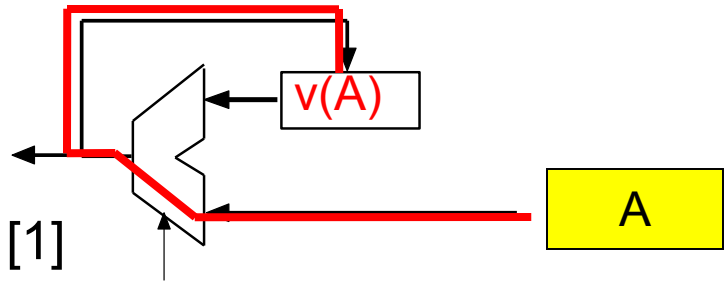
// C ← A - B

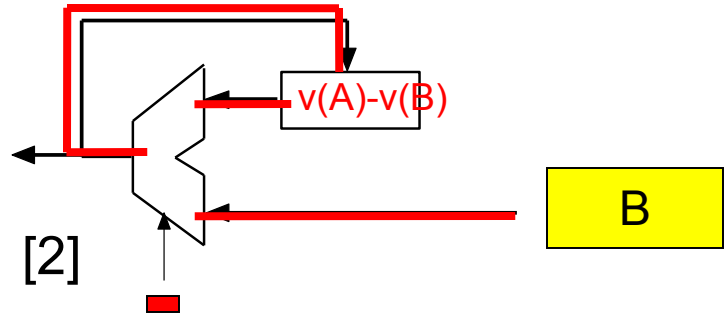
[1]	PUSH	A
[2]	PUSH	B
[3]	SUB	
[4]	POP	C

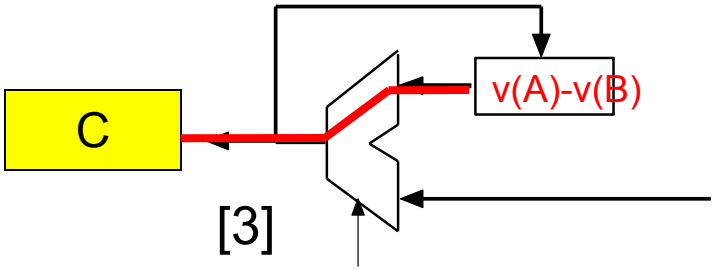


Les Machines à Accumulateur










```
[1]     LOAD  A
[2]     SUB   B
[3]     STORE C
```

Types d'instructions

- Traitement de données (Data Processing)
  - Arithmétique
  - Logique
- Transfert de données (Data transfer)
  - Mémoire / registre
  - Registre / registre
- Contrôle (Control)
  - Tests, branchements, …
- Transfert E/S (I/O Transfer)

Types d'opérandes

A ce bas niveau : nb réduit de types

- Adresses (voir plus loin)
- Scalaires
  - Entiers
  - « réels » virgule fixe
  - « réels » virgule flottante
- Caractères
  - ASCII, unicode…
- Tableaux de bits
  - Words, longs…

Adressage mémoire

- Deux problèmes:
  - Comment interpréter les adresses mémoire ?
    Endianness ?
    Alignement ?
    Ordre des bits ?
  - Comment les spécifier ?
    Modes d'adressage ?

Interprétation

Quel est l'objet sélectionné en fonction d'une adresse et d'une longueur ?

- Endianness (Big-endian / little-endian)
  - Big-Endian: MSB à la plus petite adresse
  - Little-endian: LSB à la plus petite adresse
- Alignement : l'accès à un objet de taille s octets à l'adresse A est aligné si A mod s = 0
- Ordre des bits (bit 0 à droite ou à gauche)

Modes d'adressage

Compromis entre

- Souplesse du mode d'adressage
- Complexité de calcul de l'adresse

- Adressage immédiat
  - L'instruction contient la valeur

    ex 68hc711: LDAA #5
    MAM: LOAD #5
    ATmega2560: LDI R17, 20

- Adressage direct
  - L'instruction contient l'adresse où chercher/ranger la donnée

    ex 68hc711: LDAA 5
    MAM: LOAD $1000
    Atmega2560: LDI R6, 0x1000

Modes d'adressage

- Adressage indirect
  - L'instruction se réfère à un registre dont le contenu servira d'adresse

    ex 68711: LDAA ,X
    MAM: LOAD 0,X
    Atmega2560: ST Y, R5

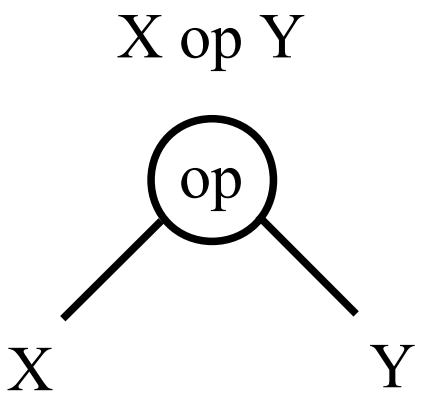
- Adressage indirect avec déplacement constant
  - L'instruction se réfère à un registre et à une constante dont la somme servira d'adresse

    ex 68711: LDAA 3,X
    Atmega2560: LDD R2,Y+10

Spécificité des machines à pile

Evaluation des expressions très efficace

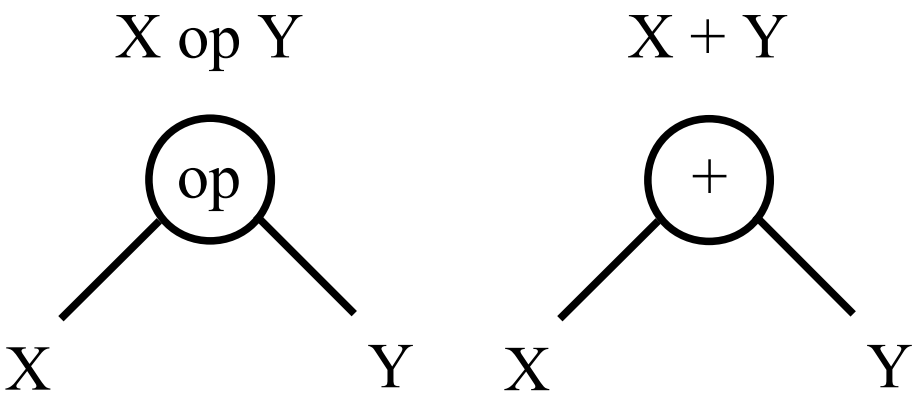
notion **d'arbre d'évaluation**



Spécificité des machines à pile

Evaluation des expressions très efficace

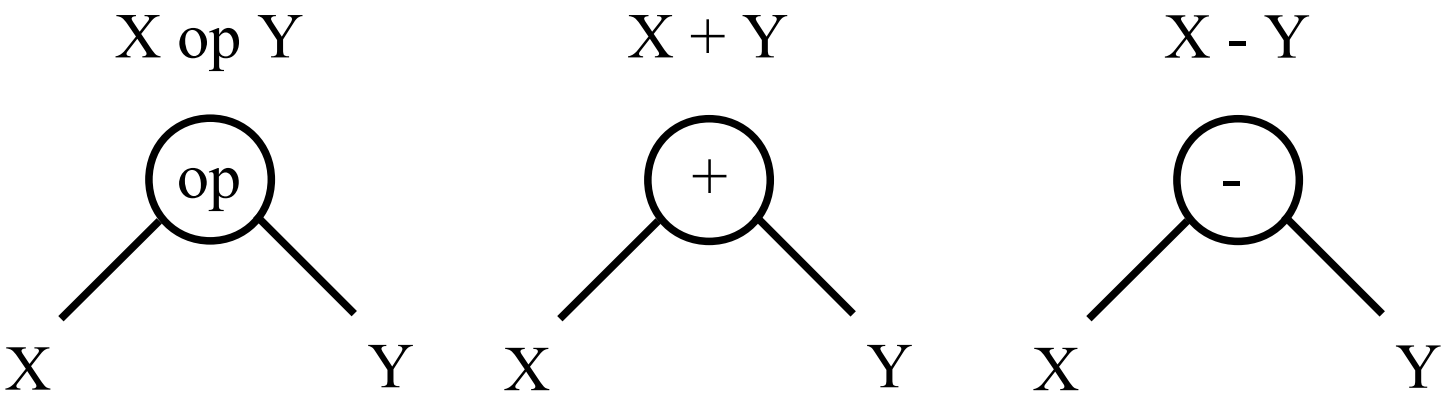
notion **d'arbre d'évaluation**



Spécificité des machines à pile

Evaluation des expressions très efficace

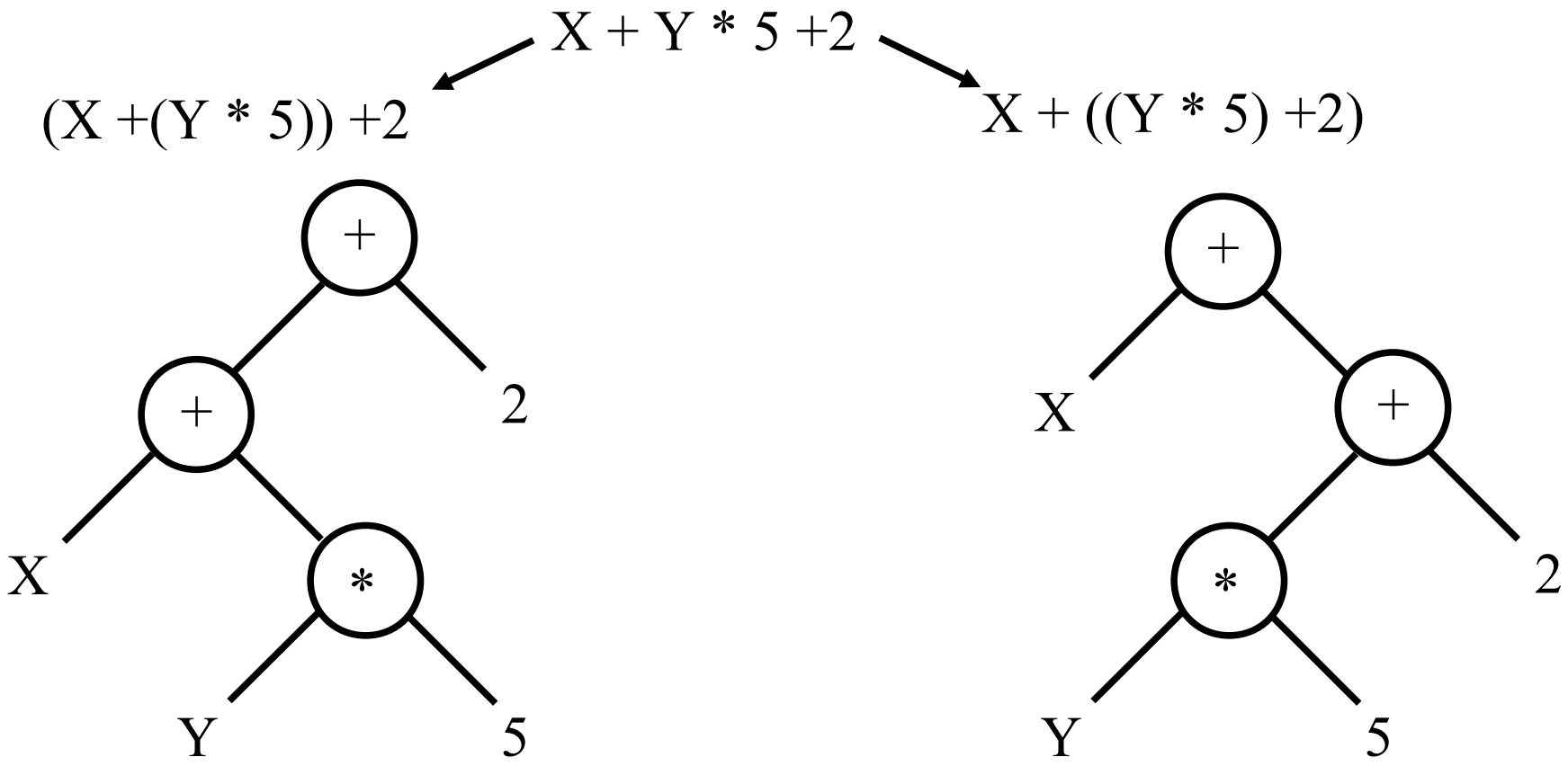
notion **d'arbre d'évaluation**



Pas forcément symétrique ...

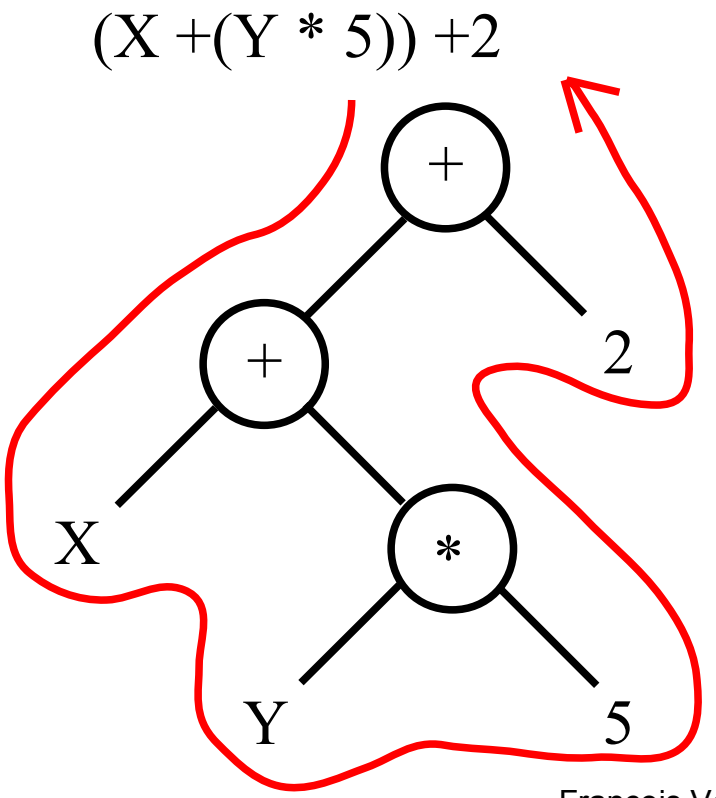
Spécificité des machines à pile

arbre d'évaluation



Spécificité des machines à pile

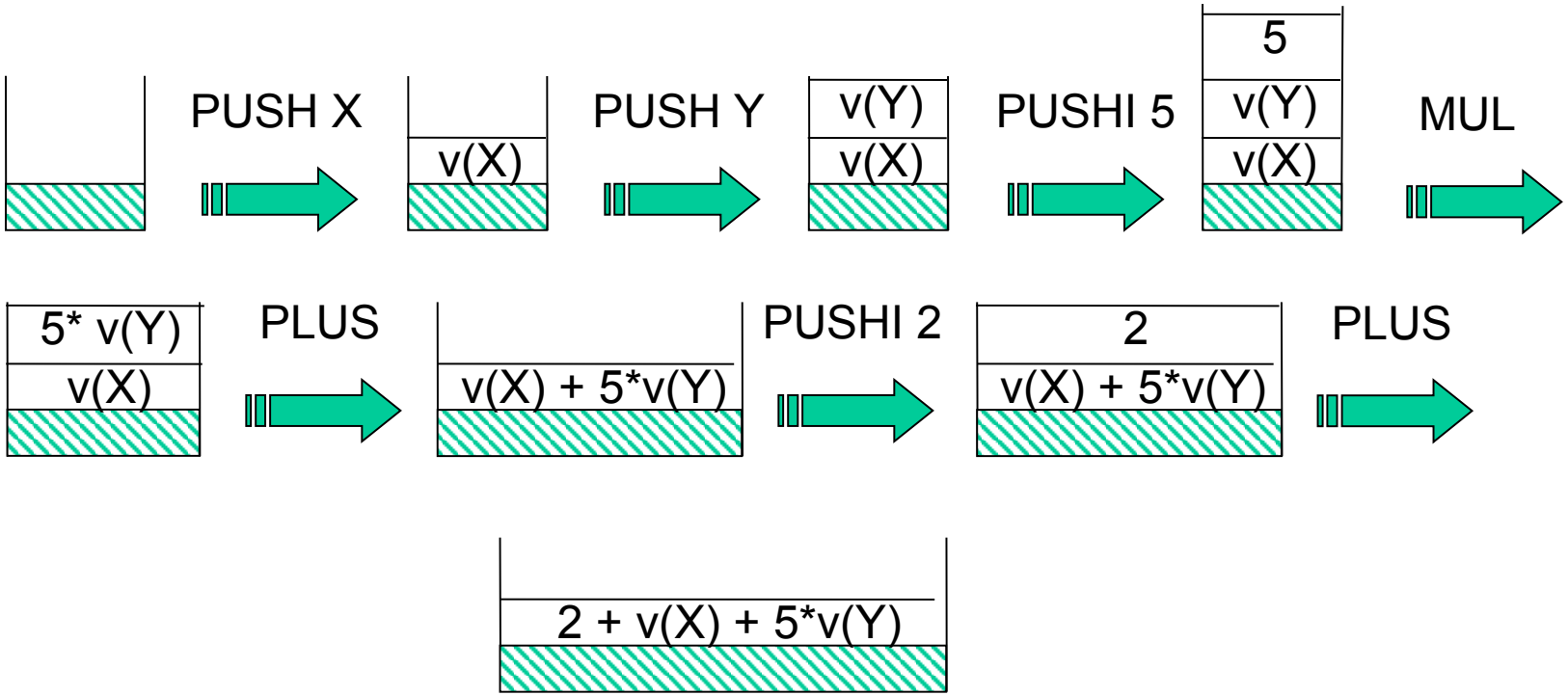
arbre d'évaluation




```
PUSH X
PUSH Y
PUSHI 5
MUL
PLUS
PUSHI 2
PLUS
```

Spécificité des machines à pile

arbre d'évaluation



Machine à accumulateur

E= ((y * 5)+x) + 2

avec x,y et E rangés respectivement
aux adresses 0, 1 et 2 en mémoire...

Machine à accumulateur

1) Comment une machine à accumulateur évalue-t'elle une expression ?

2) Existe-t'il, comme pour la machine à pile, un algorithme permettant d'optimiser les calculs ?

Machine à accumulateur

E= ((y*5)+x) + 2

avec x,y et E rangés respectivement aux adresses 0, 1 et 2 en mémoire ...

```
LOAD 1      (y) mode d'adressage ?
MULI 5      mode d'adressage ?
ADD 0       (x)
ADDI 2
STORE 2
```

Principe:

l'accumulateur contient en permanence un des opérandes résultat d'un calcul précédent et qui va servir à son tour pour le calcul suivant...

Autre exemple

E= (x+1) - (5 * y)

avec x,y et E rangés respectivement
aux adresses 0, 1 et 2 en mémoire...

Autre exemple

E= (x+1) - (5 * y)

avec x,y et E rangés respectivement
aux adresses 0, 1 et 2 en mémoire ...

LOAD 0	LOAD 3
ADDI 1	SUB 4
STORE 3	STORE 2
LOADI 5	
MUL 1	
STORE 4	5 cases mémoire ?

Autre exemple

E= (x+1) - (5 * y)

La case « E » peut servir de variable intermédiaire...

LOAD 0	LOAD 2
ADDI 1	SUB 3
STORE 2	STORE 2
LOADI 5	
MUL 1	
STORE 3	4 cases mémoire ?

Autre exemple

E= (x+1) - (5 * y)

Mais nous pouvons aussi réorganiser complètement les calculs ...

LOADI 5
MUL 1
STORE 2
LOAD 0
ADDI 1
SUB 2
STORE 2

3 cases mémoire !

Méthode systématique

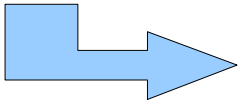
Existe-t'il une méthode systématiquement applicable

pour gérer au mieux l'accumulateur ?

Méthode systématique

Existe-t'il une méthode systématiquement applicable

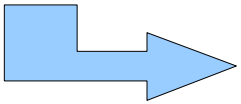
pour gérer au mieux l'accumulateur ?



Algorithme de rétro-calcul !

Algorithme rétro-calcul

l'accumulateur contient en permanence une des opérandes résultat d'un calcul précédent
et
qui va servir à son tour pour le calcul suivant...

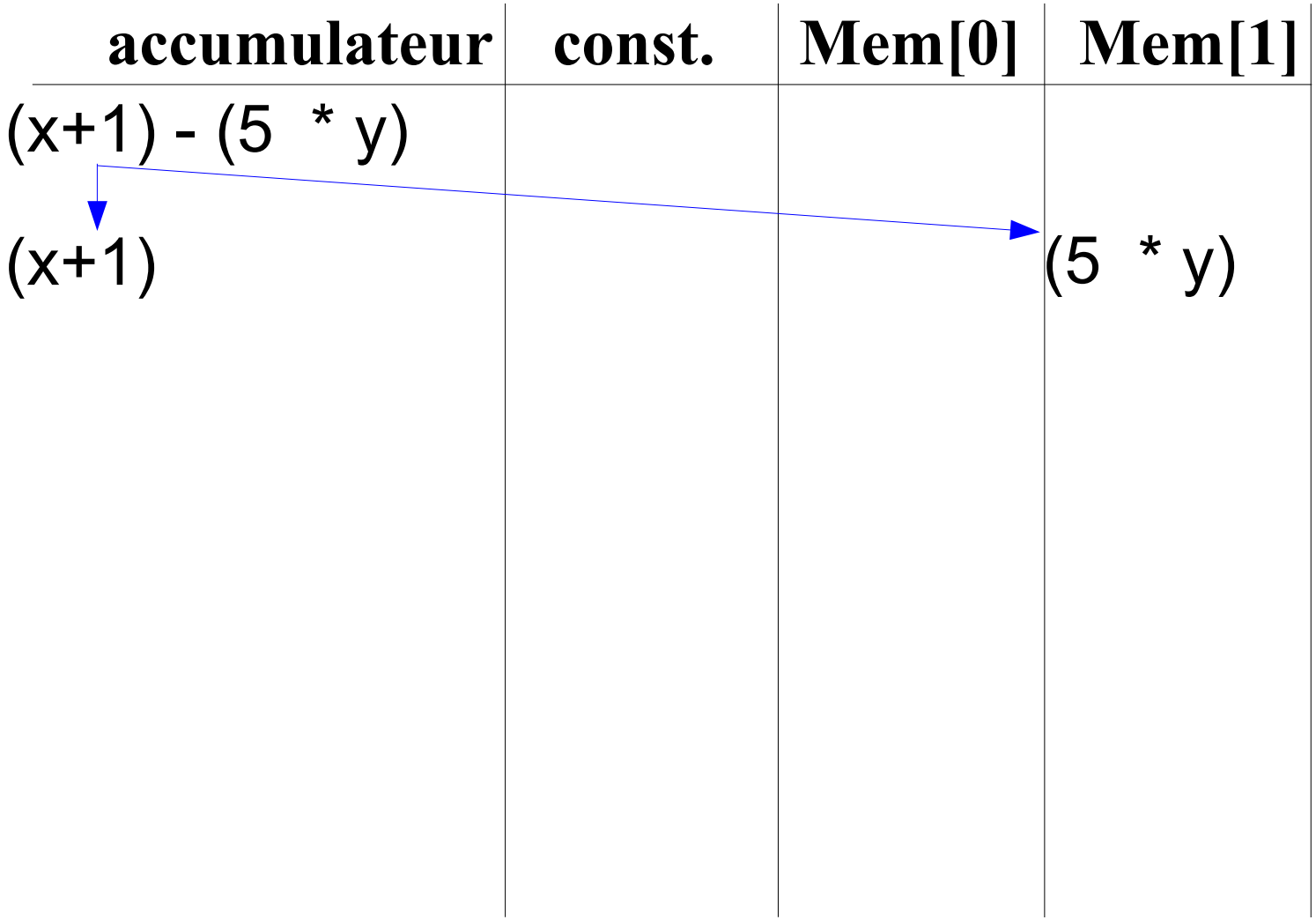


Partir du résultat final puis remonter!

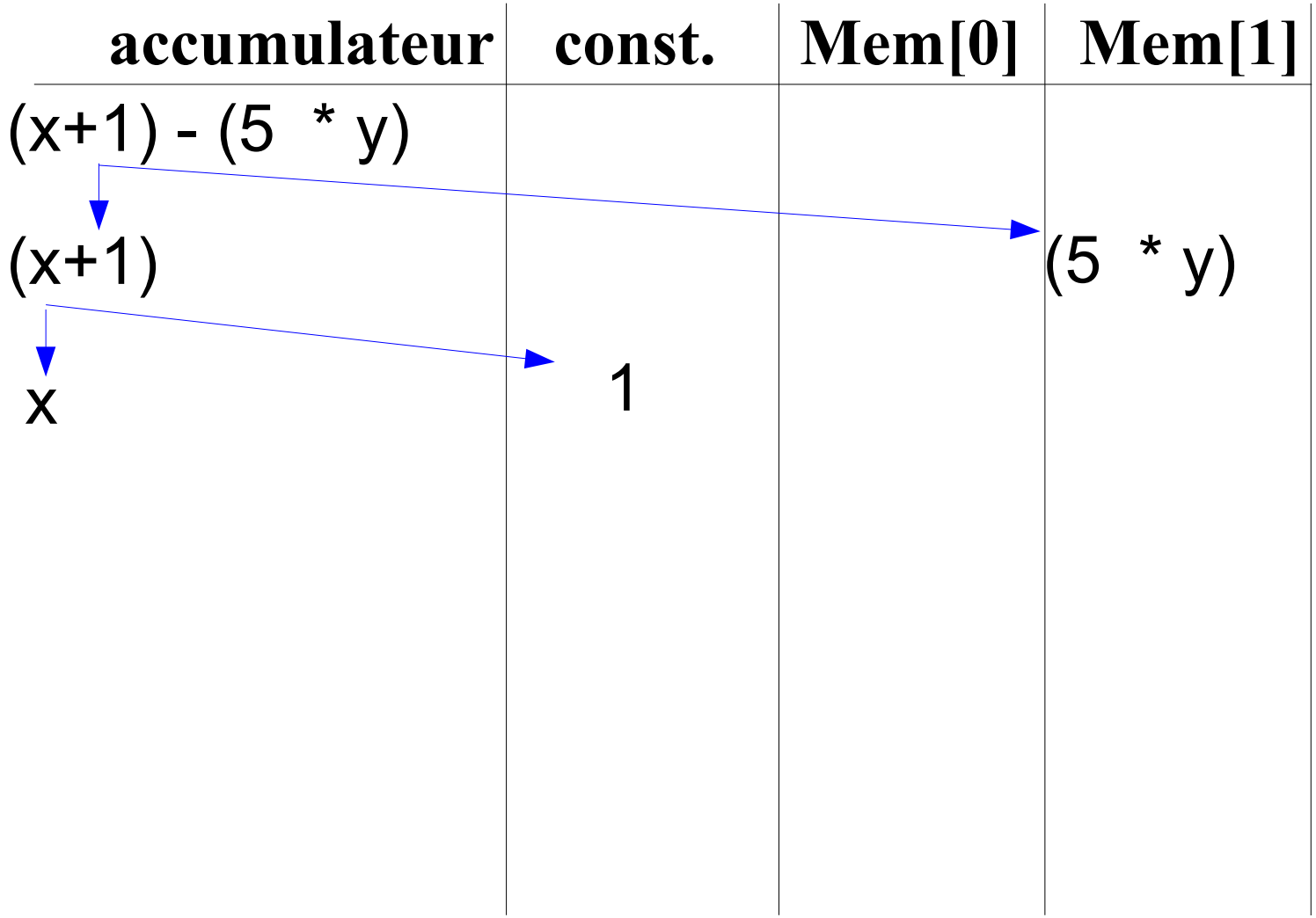
Algorithme rétro-calcul

accumulateur	const.	Mem[0]	Mem[1]
(x+1) - (5 * y)			

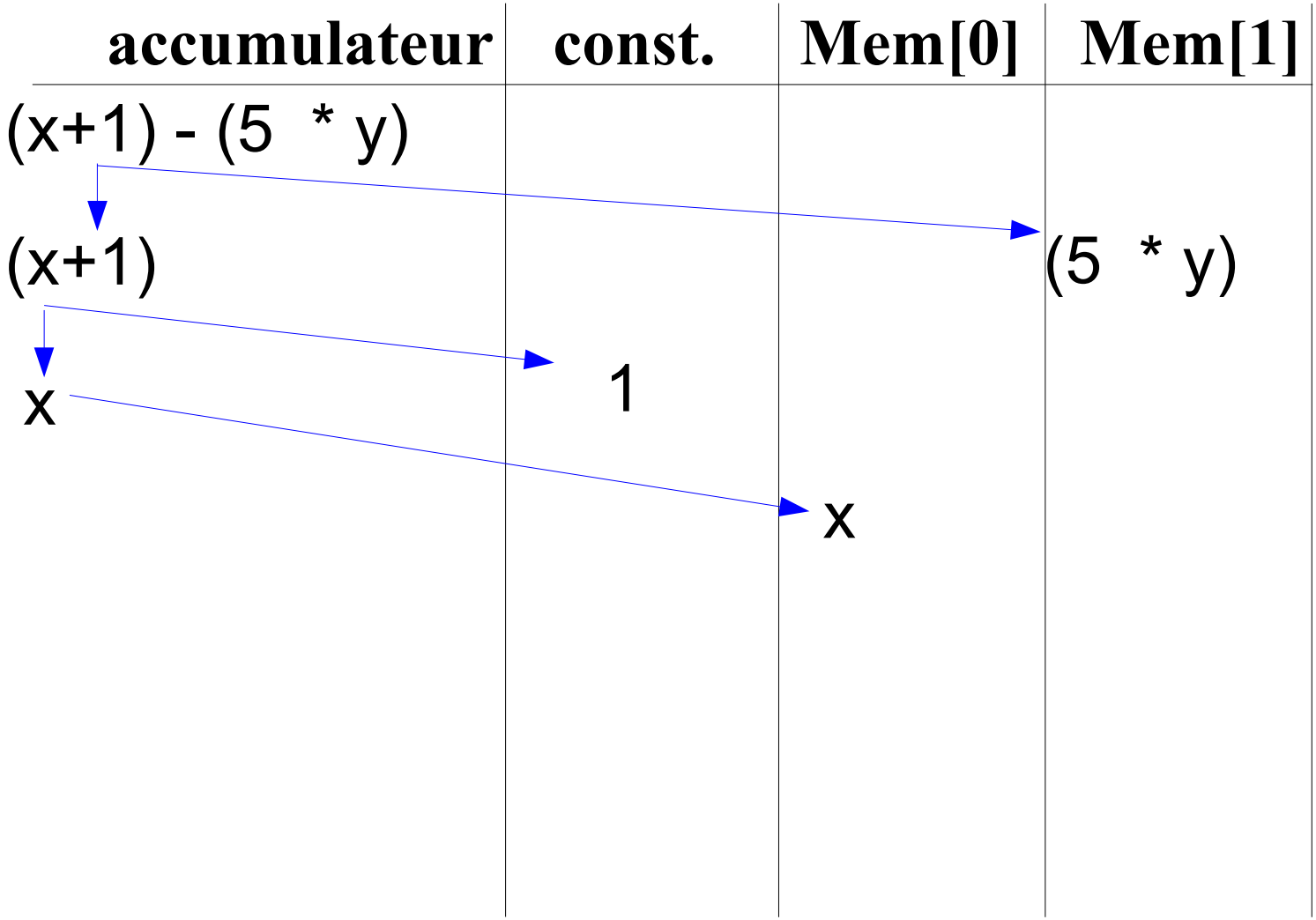
Algorithme rétro-calcul



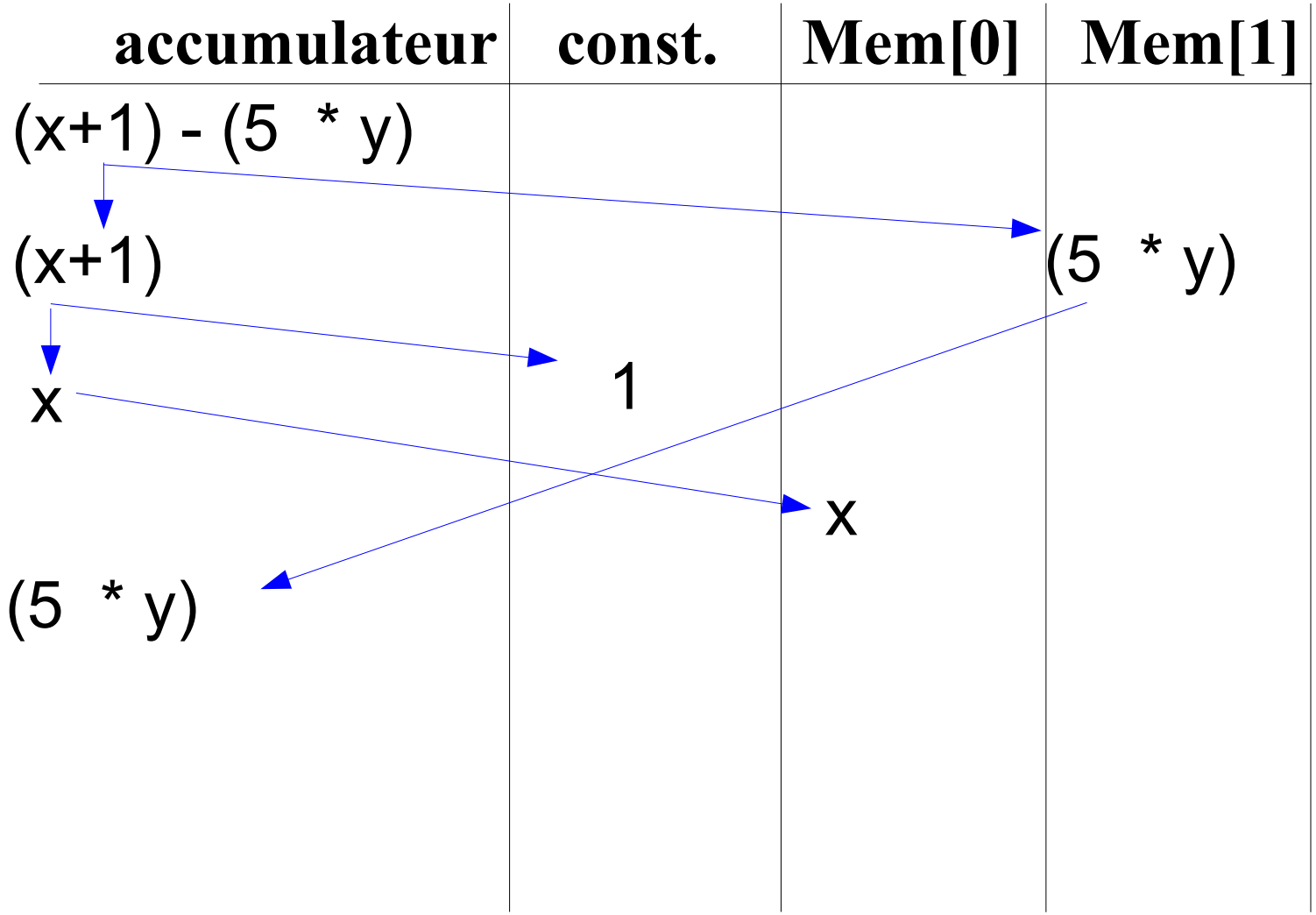
Algorithme rétro-calcul



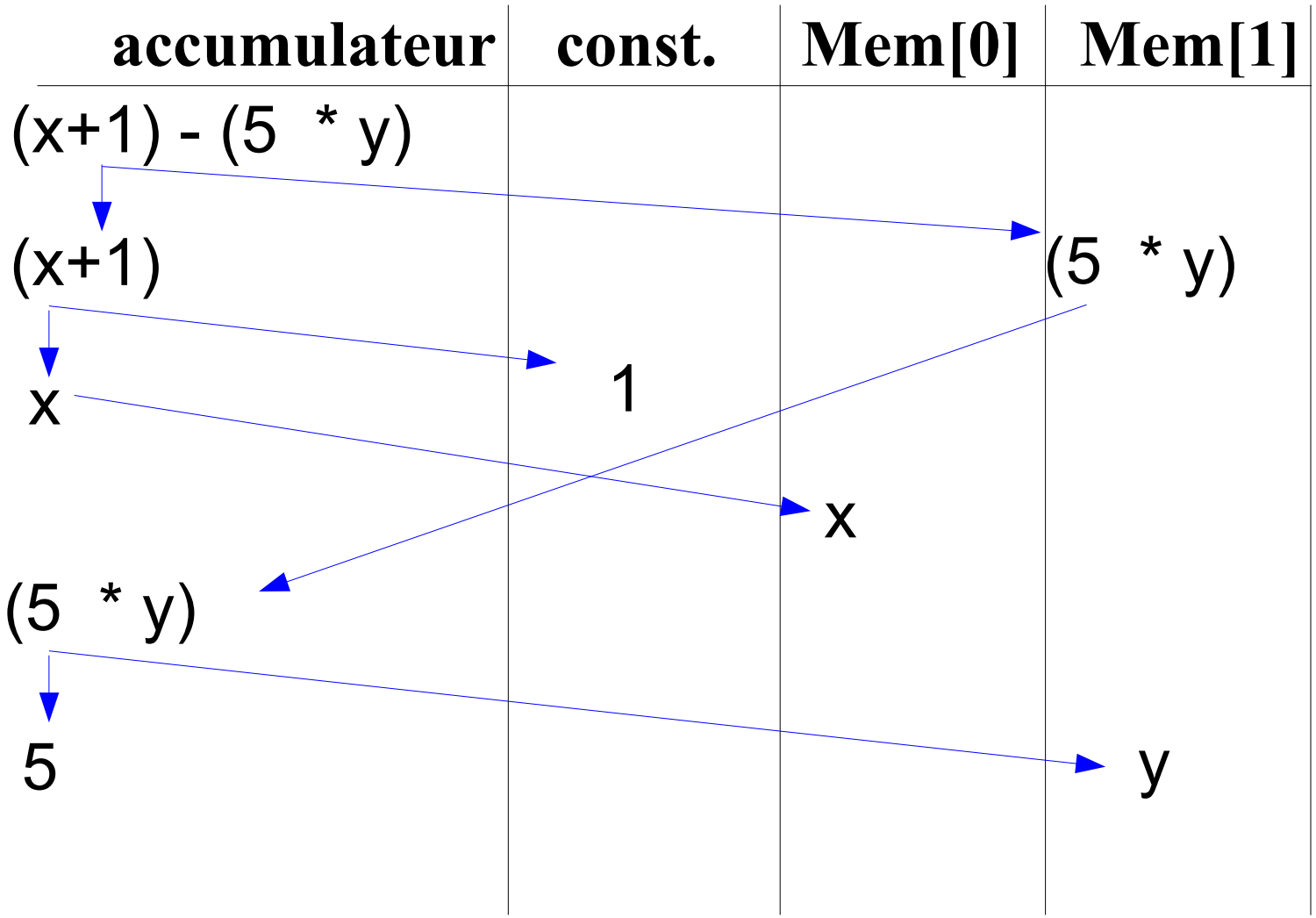
Algorithme rétro-calcul



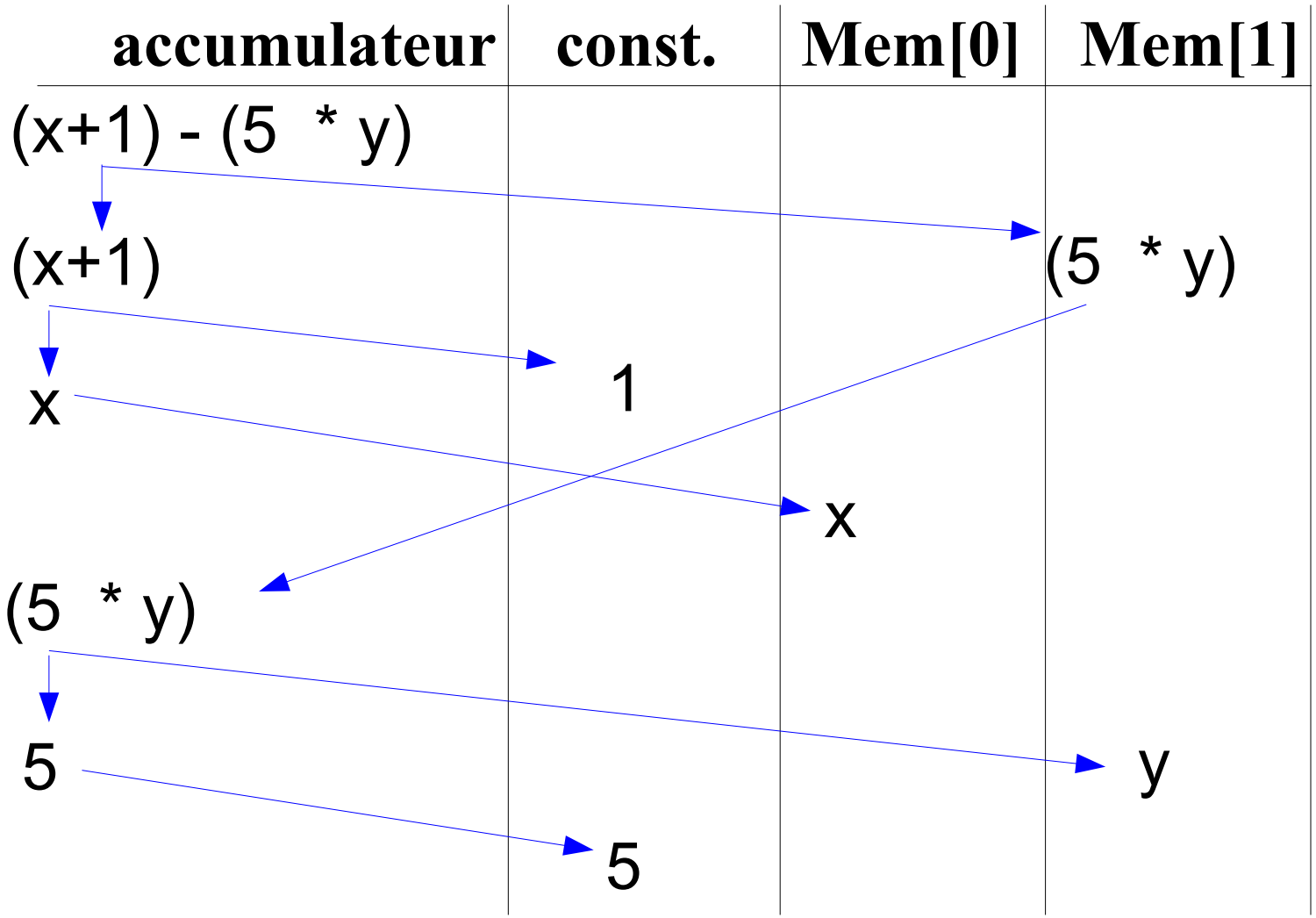
Algorithme rétro-calcul



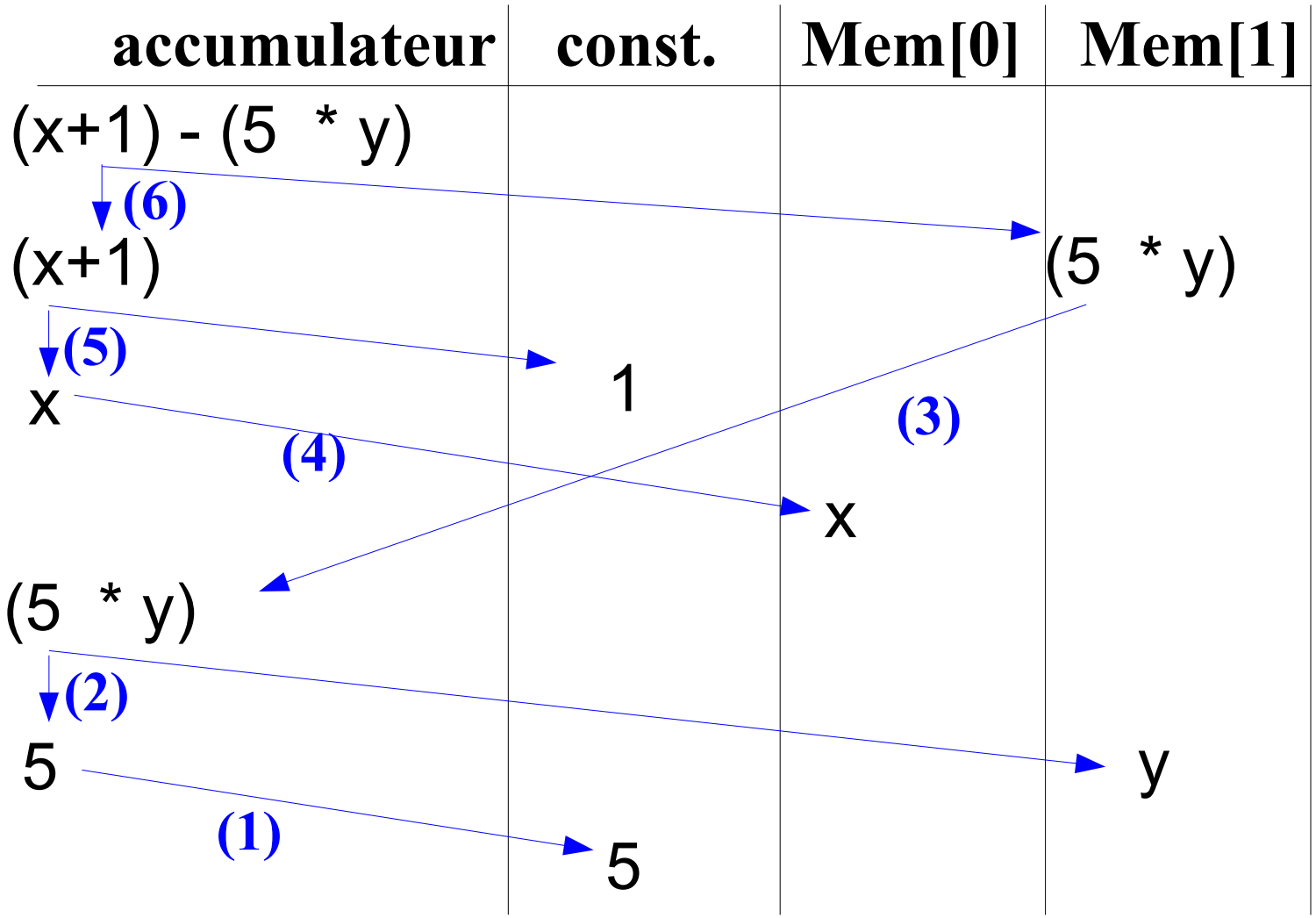
Algorithme rétro-calcul



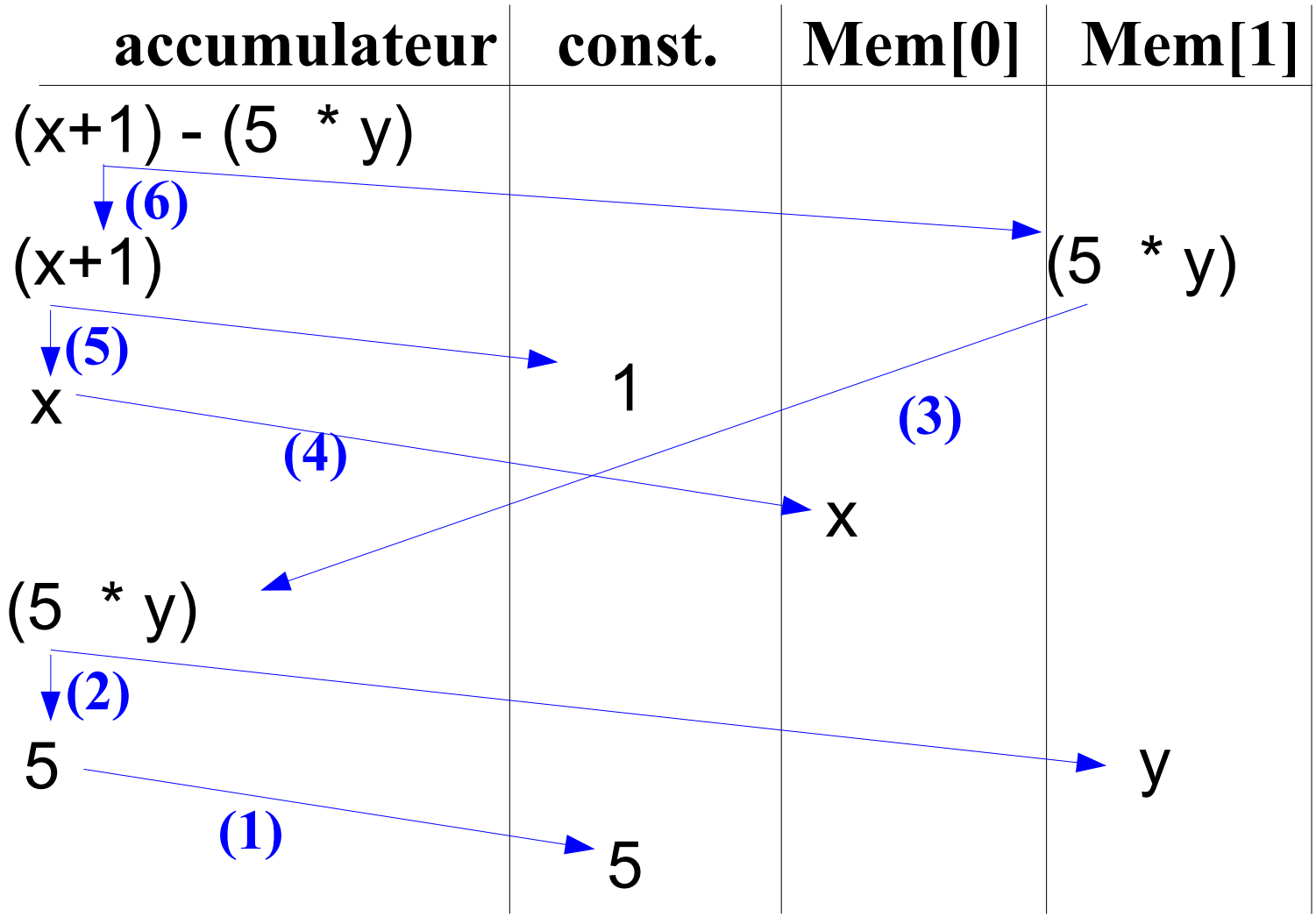
Algorithme rétro-calcul



Algorithme rétro-calcul



Algorithme rétro-calcul



(1) LOADI 5

(2) MUL 1

(3) STORE 1

(4) LOAD 0

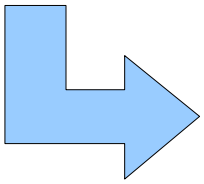
(5) ADDI 1

(6) SUB 1

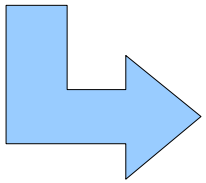
Algorithme de rétro-calcul

- $E = (x + (z - 2) * 4) + y*3$
- $E = b^2 - 4*a*c$

Conclusion



Chaque architecture possède sa propre politique d'évaluation



Et par conséquent des optimisations complètement différentes...